GBC073 · INTELIGÊNCIA COMPUTACIONAL · DESAFIO 3

O melhor treino para o nosso problema

Otimizadores e agendas de η num problema-proxy do projeto, e o que aprendemos tentando fazer um agente de RL jogar Plants vs. Zombies

Gustavo Luis de Siqueira Nascimento · Pablo Rodrigues Cardoso Araújo · 14/10/2026

Um problema supervisionado tirado do nosso jogo

Partidas
da heurística Econ5, com 15% de ações aleatórias

→

Dataset
(estado, ação, retorno G), 211 mil amostras de treino

→

CNN
85.830 parâmetros: gramado 20 × 5 × 9 → 226 valores Q

→

Perda
Huber entre Q(s, a) e G, só na ação tomada

Por que um proxy, e não o DQN direto

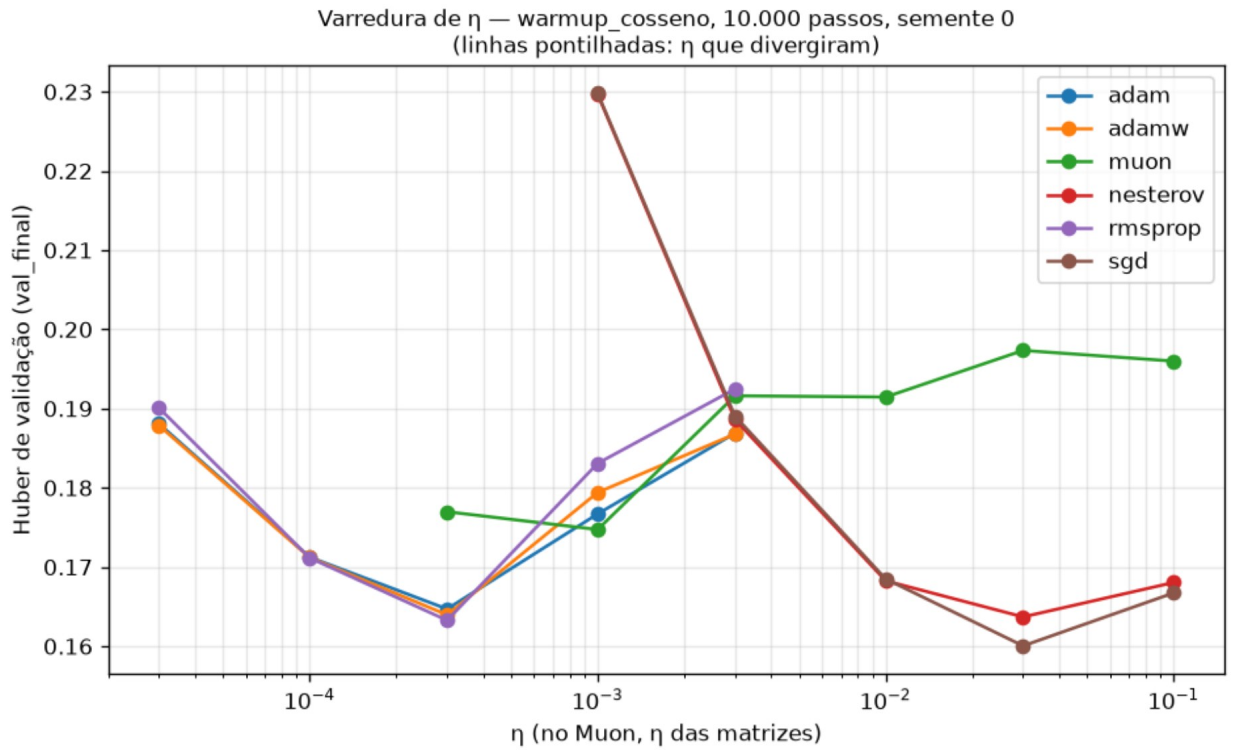
No RL, o efeito do otimizador se mistura com a exploração e com um alvo que se move. No proxy, o alvo é fixo e igual para todos: a comparação fica justa.

Como rodamos

10.000 passos, lote 256, clip 10 para todos. Varredura de η → 4 agendas → 3 finalistas + Muon com 5 sementes → teste uma vez. Vence quem supera 2 desvios combinados.

ETAPA 1 · VARREDURA DE H

Cada otimizador tem o seu η ótimo



SGD e Nesterov acertam em $3 \cdot 10^{-2}$. **Adam, AdamW e RMSprop**, em $3 \cdot 10^{-4}$.

Comparar todos no mesmo η seria injusto: em 10^{-3} o SGD é o pior; em $3 \cdot 10^{-2}$, o melhor.

O **Muon** fica acima dos outros em quase toda a faixa; o melhor dele é $\eta = 10^{-3}$ nas matrizes.

ETAPA 3 · CONFIRMAÇÃO COM 5 SEMENTES

Três finalistas empatadas, todas acima da baseline

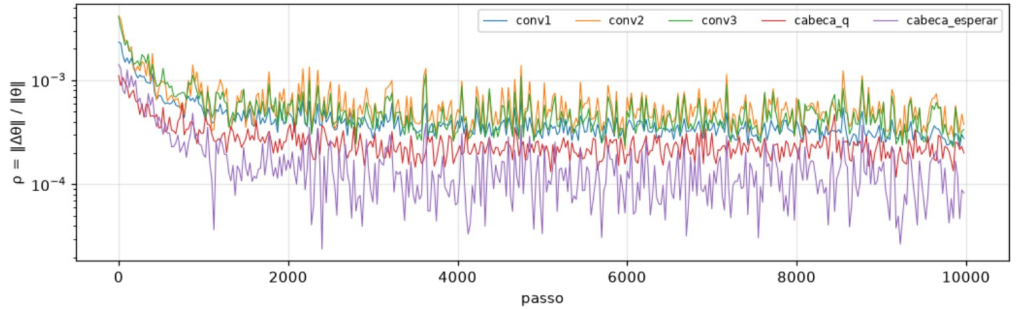
Configuração	val_final	Δ	Passos até 0,1676	Veredito
Baseline: Adam 1e-4, constante	0,1676 ± 0,0008	—	8.000–10.000	referência
SGD 3e-2, cosseno	0,1623 ± 0,0022	−0,0053	~3.250	vence a baseline
SGD 3e-2, warmup + cosseno	0,1623 ± 0,0019	−0,0053	~3.500	vence a baseline
RMSprop 3e-4, warmup + cosseno	0,1623 ± 0,0014	−0,0052	~3.000	vence a baseline
AdamW 3e-4, w + c (extra)	0,1656 ± 0,0015	−0,0020	~4.250	perde para 1 finalista
Adam 3e-4, w + c (extra)	0,1661 ± 0,0016	−0,0014	~5.000	perde para as 3
Muon 1e-3, w + c	0,1750 ± 0,0003	+0,0074	nunca	perde para todas

Teste (fases inéditas, usado uma vez): vencedora 0,1344 ± 0,0034, baseline 0,1394 ± 0,0013, Muon 0,1438 ± 0,0017. A ordem se mantém; a vencedora fica no limite do critério (1,9 s_c) e o Muon perde de novo.

O DIAGNÓSTICO · RAZÃO DE ATUALIZAÇÃO P = ‖ΔΘ‖/‖Θ‖ POR CAMADA

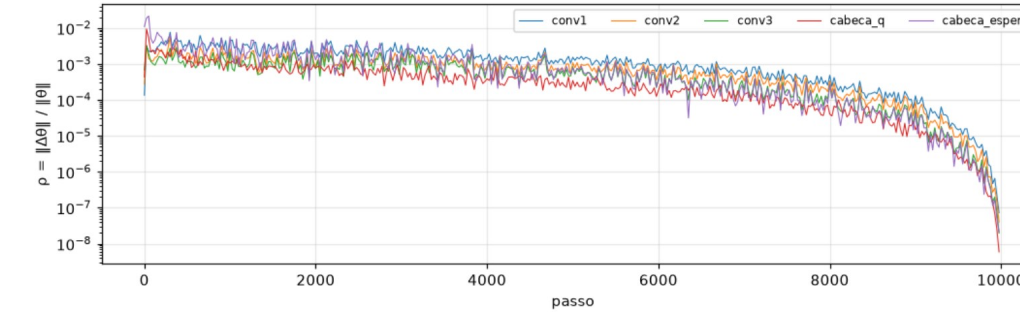
A baseline andava devagar demais

Baseline: Adam 1e-4, constante



ρ estaciona em 3–5·10⁻⁴ nas convoluções, abaixo da referência de ~10⁻³: passo pequeno.

Vencedora: SGD 3e-2, cosseno



ρ começa em ~10⁻³ e o cosseno o leva a 10⁻⁶ no fim: anda rápido no começo e assenta no vale.

‖g‖ nunca chegou ao corte (10) e não teve picos no começo: era tamanho de passo, não instabilidade; por isso o warmup não fez diferença. O Muon fez o contrário: a menor perda de treino (0,111) e a pior validação (0,175). Ele decorou o ruído do alvo.

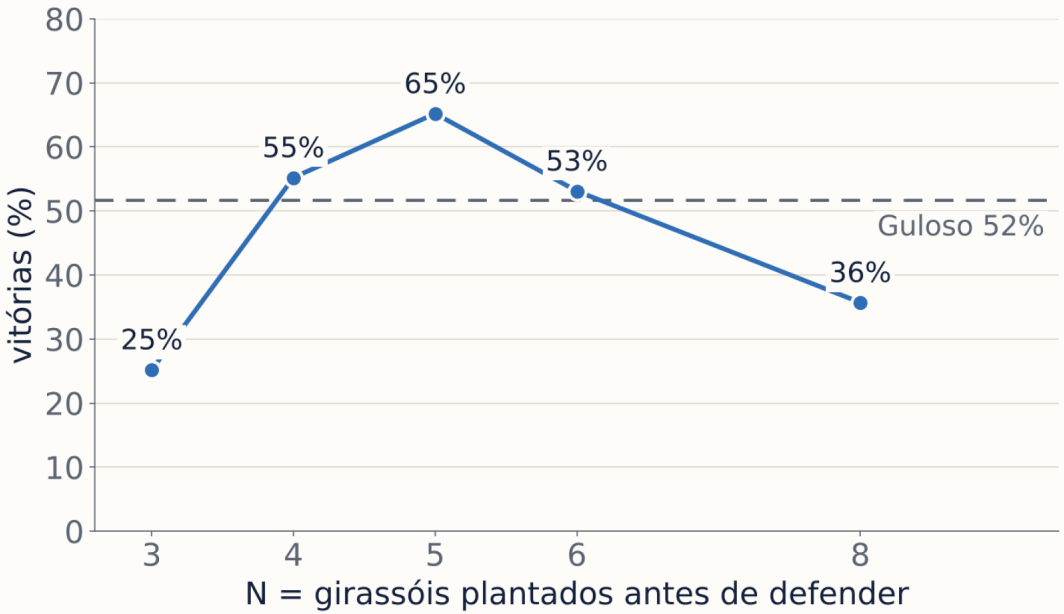
O agente decide o que plantar, onde e quando esperar

Estado: 25 canais sobre o gramado 5 × 9: plantas, vidas, zumbis, sol, recargas e onde dá para plantar.

Ações: 5 plantas × 45 células + esperar = 226, com máscara das inválidas.

Recompensa: dano causado ÷ vida total da fase, +1 na vitória, −1 na derrota.

Fases: hordas sorteadas por semente; 21 fases de treino, 6 inéditas para o teste.



O dilema existe: girassóis de menos ou de mais perdem. O ótimo fica no meio (N = 5).

PRIMEIRA ESCOLHA · DOUBLE DQN COM MÁSCARA

DQN: aprender o valor de cada jogada

Como funciona

A rede estima Q(s, a): quanto se espera ganhar fazendo a em s. O agente joga a de maior Q, só entre as válidas.

Aprende com o alvo de Bellman, r + γ·Q_alvo(s′, a′), usando replay de 100 mil transições, rede alvo copiada a cada 2.000 atualizações, ε de 1 a 0,05 e retorno de 10 passos.

Por que escolhemos

As ações são discretas, e a máscara entra direto no argmax.

É off-policy: o replay reaproveita cada partida muitas vezes.

É o padrão em jogos (Atari, Mnih et al., 2015), e era o plano do nosso CONCEITO desde o início.

Rede: CNN totalmente convolucional, 3 convoluções 3 × 3 → um mapa Q de 5 × 5 × 9 (planta × gramado) + Q(esperar).

O DQN NÃO APRENDIA · K = 10, 400 MIL PASSOS

Tentamos consertar o DQN girando botões

Tentativa	O que mudou	Vitórias
Otimizador	Adam 1e-4, constante (a baseline)	9,2% ± 7,1
Otimizador	SGD 3e-2, cosseno (a vencedora do proxy)	0%
Otimizador	Muon 1e-3 + AdamW	0,1%
Exploração	"Intencional": sorteia um tipo de planta e junta sol até poder plantá-lo	1,3% ± 1,1
Arquitetura e currículo	cnn1d, cnn2, cnn3 (dueling, contexto, residual) e fases fáceis primeiro	implementado, não rodado

Referências com k = 10: Econ5 67%, Guloso 41%. Nenhum botão mexeu no resultado, e a vencedora do proxy foi a pior: perda de valor menor não é política melhor. **Paramos e fizemos um diagnóstico**, com uma tabela de decisão escrita antes de rodar.

O DIAGNÓSTICO · Q(S, A) = V(S) + A(S, A)

A rede aprende o valor do tabuleiro, não a diferença entre jogadas

~1%

Trocar uma jogada muda o retorno em 0,006 (mediana, por clones do jogo); o retorno varia 0,62 entre estados.

98%

das vezes, adiar um plantio uma decisão não muda nada: a maioria das 300 decisões por partida é irrelevante.

0,5%

de vitórias para a rede do Desafio 3, que ajusta bem o retorno mas joga pelo argmax.

O V (quão bom é o tabuleiro) varia muito; o A (qual jogada é melhor) é ~1% disso. Entre 226 valores quase iguais, o argmax escolhe a jogada com o maior erro.

Mas, numa prova com clones, a rede treinada com dados do Econ5 ordenou até 73% dos pares de jogadas. **O problema estava no treino do RL, não na rede.**

MUDANÇA 1 · K = TICKS ENTRE DECISÕES DO AGENTE

Decidir a cada 20 ticks, não a cada 10

Baselines	k = 10	k = 20	k = 30
Econ5, vitórias	67%	65%	63%
Guloso, vitórias	41%	52%	49%
Decisões por partida	256	127	85
Pico da curva Econ(N)	N = 5	N = 5	N = 5

Metade das decisões: a vitória fica a ~150 decisões do começo, não a ~300.

Cada decisão pesa mais: adiar uma jogada passa a mudar o resultado.

O dilema continua: o pico da curva Econ(N) fica em N = 5.

$\gamma = 0{,}994 = 0{,}997^2$: o mesmo horizonte em tempo de jogo. Efeito colateral: o Guloso sobe para 52%, então a régua do agente fica mais exigente.

MUDANÇA 2 · SUGESTÃO DO PROFESSOR

PPO: aprender a política diretamente

DQN

Aprende Q(s, a) para as 226 jogadas.

Joga pelo argmax: um erro pequeno troca a jogada inteira.

Explora com ε; reaproveita dados pelo replay.

PPO

Ator: uma probabilidade por jogada; as inválidas ficam com probabilidade 0.

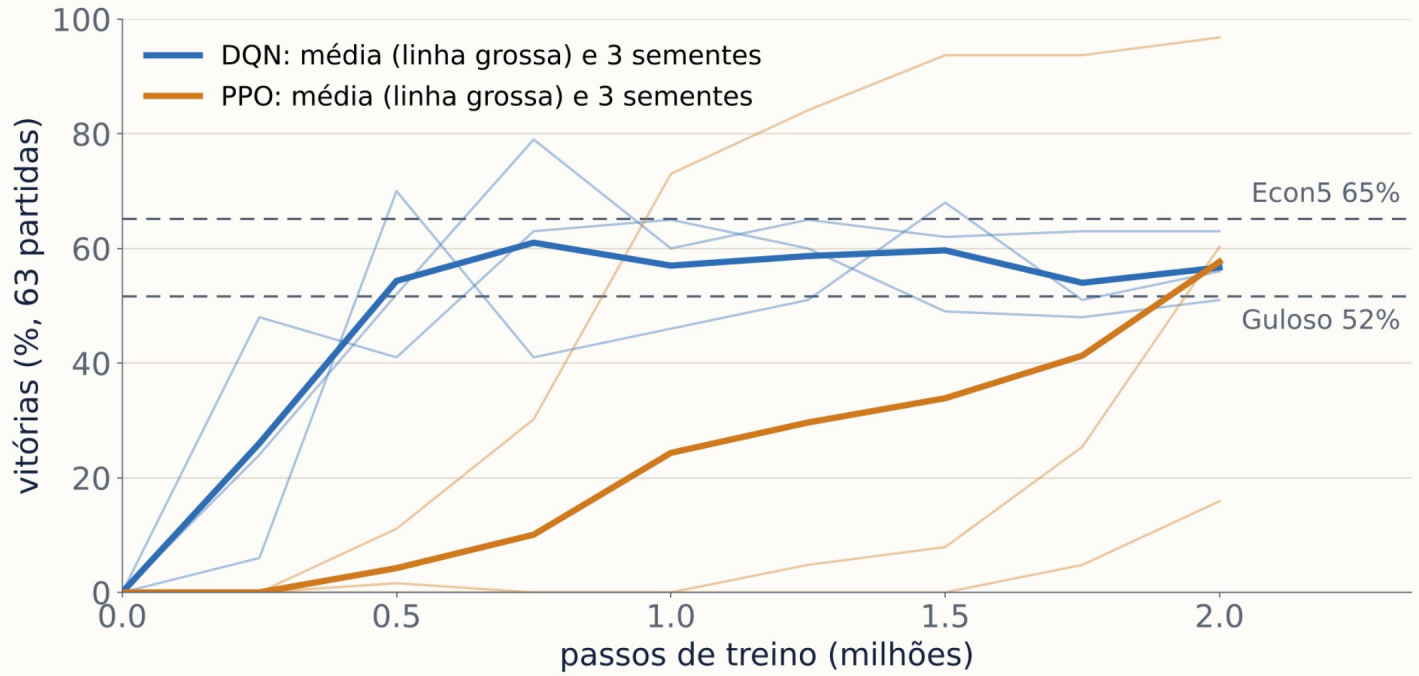
Crítico: só V(s), o que a rede já aprende bem.

Aumenta um pouco a chance das jogadas que saíram melhor que o esperado (retorno − V), com o passo limitado pelo clip. A exploração vem do próprio sorteio.

Nossa versão: a mesma cnn1d (o mapa de A vira as probabilidades, o V vira o crítico), 16 partidas em paralelo, ~1.900 passos/s: **~4× mais rápido que o DQN**.

RESULTADOS · K = 20, 2 MILHÕES DE PASSOS, SEMENTES 0–2

Com k = 20, os dois aprendem; o PPO é 4× mais rápido



DQN: 54,7% ± 10,0

PPO: 57,2% ± 38,6

Empate formal: Δ = 2,5 pontos, e o critério pedia 56.

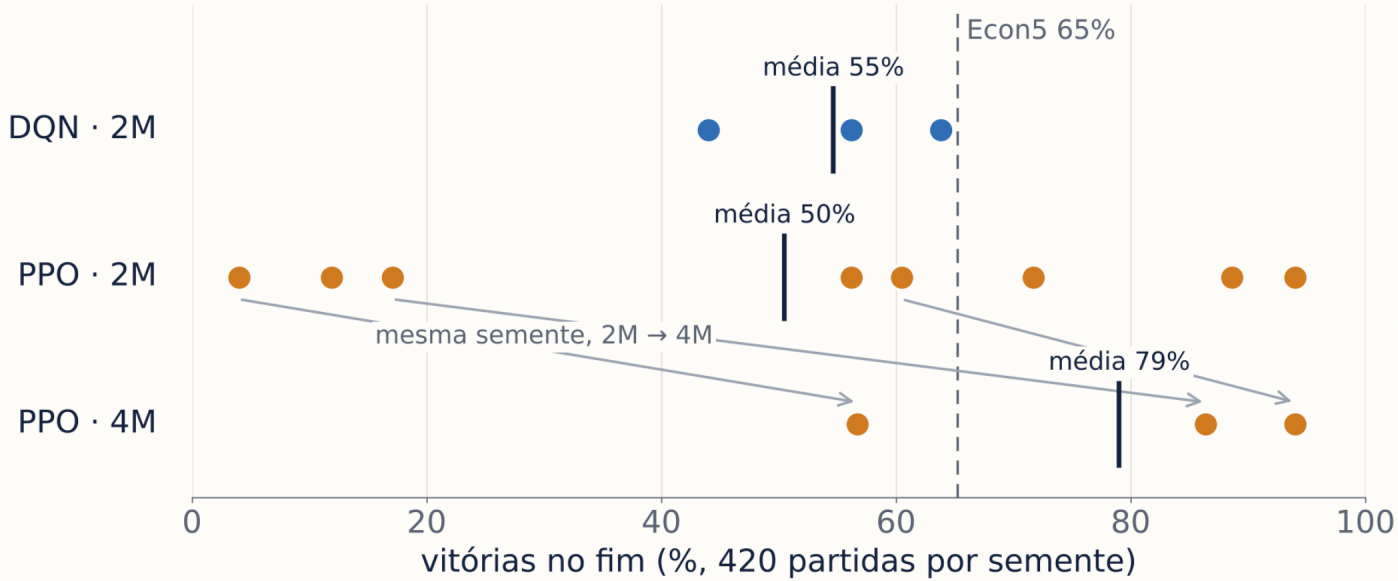
Tempo por semente: DQN 139 min, PPO 35 min.

O DQN saiu de 9% para 55% só com k = 20 e mais orçamento.

A forma das curvas difere: o DQN estabiliza perto de 60% e oscila; o PPO ainda sobe aos 2M.

ONDE ESTAMOS

O problema atual: o PPO varia muito entre sementes



PPO a 2M: de 4% a 94%; só 3 de 8 sementes passam do Econ5.

A 4M: as 3 piores foram a 57–94%, e as curvas ainda subiam. Parte disso é orçamento.

DQN: varia menos, mas chega ao pico cedo (até 82% com 0,7M) e depois piora.

CONCLUSÃO E PRÓXIMOS PASSOS

O otimizador mudou 3% no proxy. No jogo, o que destravou o aprendizado foi decidir a cada 20 ticks e treinar mais.

O que aprendemos

SGD 3e-2 com cosseno vence no proxy; a baseline tinha passo pequeno. O Muon decora o ruído do alvo.

Perda de valor menor não é política melhor: medir antes de girar botões.

PPO e DQN empatam em passos iguais; o PPO é 4× mais rápido.

Próximos passos

8 sementes do PPO a 4M (rodando esta noite) e uma longa, para achar onde a curva achata.

Teste nas 6 fases inéditas, uma única vez.

Uma heurística humana forte como baseline, e a análise de erros por clones.